

P2575R0

2022-05 Library Evolution Poll Outcomes

Published Proposal, 2022-06-22

Authors:

[Bryce Adelstein LeBach \(he/him/his\) — Library Evolution Chair](#) (NVIDIA)

[Fabio Fracassi — Library Evolution Vice Chair](#) (CODE University of Applied Sciences)

[Ben Craig — Library Evolution Vice Chair](#) (NI)

[Inbal Levi — Ranges Chair \(SG9\) and Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1	Introduction
2	Poll Outcomes
3	Selected Poll Comments
3.1	C++23 Polls
3.1.1	[P1885R10] Naming Text Encodings to Demystify Them
3.1.2	[P0792R8] <code>function_ref</code>
3.1.3	[P1223R3] <code>find_last</code>
3.1.4	[P1169R3] <code>Static operator()</code>
3.1.5	[P2553R1] Make <code>mdspan size_type</code> Controllable

- 3.1.6 [P2554R0] C-Array Interoperability Of `mdspan`
- 3.1.7 [P2540R0] Empty Product For Certain Views
- 3.1.8 [P2538R0] ADL-Proof `projected`
- 3.1.9 [LWG3646] `ranges::view_interface::size` Returns A Signed Type
- 3.1.10 [P2520R0] `move_iterator` Should Be A Random Access Iterator
- 3.1.11 [P2499R0] `string_view` Range Constructor Should Be Explicit
- 3.1.12 [P2549R0] `unexpected` Should Have `error` As Member Accessor
- 3.1.13 [P2517R0] Add A Conditional `noexcept` Specification To `apply`
- 3.2 C++26 and Technical Specification Polls
 - 3.2.1 [P2300R5] `std::execution`
 - 3.2.2 [P1083R5] `resource_adaptor`
 - 3.2.3 [P1202R4] Asymmetric Fences

References

Informative References

§ 1. Introduction

In 2022-05, the C++ Library Evolution group conducted a series of electronic decision polls [\[P2574R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 34 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
------	----	----	---	----	----	---------

Poll 1.1: Return [P1885R10] Naming Text Encodings to Demystify Them to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	6	11	2	0	1	Consensus in favor.
Poll 1.2: Return [P0792R8] function_ref to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	11	10	1	1	1	Consensus in favor.
Poll 1.3: Return [P1223R3] find_last to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	13	9	1	1	0	Consensus in favor.
Poll 1.4: Send [P1169R3] Static operator() to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	13	9	0	0	0	Unanimous consensus in favor.
Poll 1.5: Send [P2553R1] Make mdspan size_type Controllable to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	7	9	1	1	0	Consensus in favor.
Poll 1.6: Send [P2554R0] C-Array Interoperability Of mdspan to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	6	10	3	0	0	Strong consensus in favor.
Poll 1.7: Send [P2540R0] Empty Product For Certain Views to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	7	7	2	2	1	Weak consensus in favor.

Poll 1.8: Send [P2538R0] ADL-Proof projected to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	8	11	2	0	0	Strong consensus in favor.
Poll 1.9: Send the proposed resolution in [LWG3646] <code>ranges::view_interface::size</code> Returns A Signed Type to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	10	9	0	0	0	Unanimous consensus in favor.
Poll 1.10: Send [P2520R0] <code>move_iterator</code> Should Be A Random Access Iterator to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	12	5	1	0	0	Strong consensus in favor.
Poll 1.11: Send [P2499R0] <code>string_view</code> Range Constructor Should Be Explicit to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	13	7	1	2	0	Consensus in favor.
Poll 1.12: Send [P2549R0] <code>unexpected</code> Should Have error As Member Accessor to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	9	9	2	0	0	Strong consensus in favor.
Poll 1.13: Send [P2517R0] Add A Conditional noexcept Specification To apply to Library Working Group for C++23, classified as an improvement	14	8	0	0	0	Unanimous consensus in favor.

of an existing feature ([P0592R4] bucket 2 item).						
Poll 2.1: Send [P2300R5] <code>std::execution</code> to Library Working Group for C++26, classified as a focus ([P0592R4] bucket 1 item).	12	6	2	2	3	Weak consensus in favor. The Chair, Bryce Adelstein Lelbach, asked Vice Chairs Fabio Fracassi and Ben Craig to determine consensus on this poll, as the Chair is one of the co-authors of P2300.
Poll 2.2: Return [P1083R5] <code>resource_adaptor</code> to Library Working Group for C++26, classified as an addition ([P0592R4] bucket 3 item).	3	7	1	4	2	No consensus. The paper was not updated to follow Library Evolution guidance, which was the cause of most of the votes against. The authors will be asked to produce a new revision with the requested changes and then we will conduct another vote.
Poll 2.3: Send [P1202R4] Asymmetric Fences to Library Working Group for the Concurrency TS v2, classified as an addition ([P0592R4] bucket 3 item).	6	8	1	1	0	Consensus in favor.

§ 3. Selected Poll Comments

§ 3.1. C++23 Polls

§ 3.1.1. [\[P1885R10\]](#) Naming Text Encodings to Demystify Them

We need this for proper portability

— Strongly Favor

Improving unicode support is a very important endeavour for C++

— Strongly Favor

It's clearly a useful feature. I was being held back by the question of what to do when `CHAR_BIT != 8`, but I found an answer in the proposal. I'm unsure the proposed resolution to that issue is ideal, but it's workable so I can do with it.

— Weakly Favor

My objections to the prior version of the paper have been addressed by restricting the scope to narrow encodings.

— Weakly Favor

The removal of the wide encodings (which were really still byte encodings) remedies the major issues with the previous revision.

— Weakly Favor

I remain dissatisfied with the incoherently capitalized constants. (Can someone explain why the T in "Unknown8BiT" is capitalized?)

— Neutral

Useful, but I'm a bit concerned about the implementation burden for implementation that support multiple platforms. Much too late to review this in time for C++23 though.

— Strongly Against

§ 3.1.2. [\[P0792R8\]](#) `function_ref`

Useful functionality especially when one doesn't want to convert otherwise-non-template code to template code just to accept an invocable.

— Strongly Favor

This is directly useful to my customers, who tend to roll out their own version in practice.

— Strongly Favor

This is an essential building block that has widespread industry use - we shouldn't fail to standardize existing practice. Especially if it is such a handy low-cost abstraction.

— Strongly Favor

I suppose it's fine to get this in now in this form, but the enhancements in p2472r look very attractive.

— Weakly Favor

The adjustments for prvalue initializers have made this facility as good as the best of the public implementations of similar features, although dangling is still possible by assigning to, say, a `std::function_ref` member variable.

— Weakly Favor

I have mixed feeling. History tells us non-trivially copy-able types are generally less useful, but only supporting function pointer is limiting in terms of API, and i don't know if we have enough visibility on the recent changes

— Neutral

Uncertain whether we've ended up with the right design, after a lot of changes and footgun removal. I think it's too late to be sure about it, would be safer to aim for C++26.

— Weakly Against

Disallowing member pointers is inconsistent with almost everything we've added to the standard library since C++11; see, e.g., `function`, `bind(/_front/_back)`, `move_only_function`, `reference_wrapper`, `not_fn`, `apply`, `thread`, all the ranges algorithms and views, etc.

Sure, to maintain the size at ≤ 2 pointers would require storing (at least PMFs) by reference, and hence lead to potential dangling in the case of misuse. But so does using a simple lambda. This feature has sharp edges by design; banning member pointers doesn't meaningfully reduce them, only adds a gratuitous inconsistency.

And if we want to reduce dangling errors, why aren't we storing sufficiently small trivially copyable types by value?

— Strongly Against

§ 3.1.3. [\[P1223R3\]](#) `find_last`

It clears many back and forth arithmetics on the paper when using reversed iterators.

— Strongly Favor

`find_last()` is clear compared to the code required without it.

— Strongly Favor

We have this already as e.g. a `std::string` member function. It is generally useful, so should be provided.

— Strongly Favor

Support for sentinels is already an improvement over the `reverse_iterator` approach (which would have to convert them first); returning the end iterator is an additional aspect that would be cumbersome to reimplement.

— Weakly Favor

I'm not super convinced by the return type change (this seems similar to the catamorphic algorithms for which we don't usually return the past-the-end iterator despite necessarily computing it - see e.g., min/max/minmax, count, etc.), but not enough to vote against.

— Weakly Favor

Whilst `find_last` enables no new functionality, the paper does a good job illustrating the inherent complexity of using `reverse_iterator`. This “new” algorithm vastly improves readability. Following the principle of useful returns is a good reason to introduce a return type inconsistency, even if the supplied information is rarely used.

— Weakly Favor

Too late, no time to review it.

— Weakly Against

§ 3.1.4. [\[P1169R3\]](#) `Static operator()`

Enabling additional optimizations by removing redundant instructions is a big win to me. Unfortunately this has to be an explicit opt-in, which prevents a slew of existing functors from exploiting this. The mentioned library changes (deduction guides) must also be applied to `function_ref` and `move_only_function`!

— Strongly Favor

The benefits are direct for the people I work with. There have been questions about per-compiler codegen on the reflectors, but I think our vendors can find solutions were there were potential issues.

— Strongly Favor

It was always an unnecessary restriction to prevent call operators from being static.

— Strongly Favor

If `operator()` doesn't require an object it shouldn't need to be non-static.

— Strongly Favor

The language feature is useful, even if it adds another wrong default, and the library support for it is straightforward and correct (although `std::function_ref` may need work for it).

— Weakly Favor

I've wanted this for years. The proposal seems solid. Won't take much time to review.

— Weakly Favor

It's unfortunate that we can't make lambdas handle this automatically, but I still want this and the potentially better code generated as a result.

— Weakly Favor

§ 3.1.5. [\[P2553R1\]](#) Make `mdspan size_type` Controllable

Thanks for measuring real-world programs.

— Strongly Favor

The rationale makes sense, and the proposal is based on real-world experience from the expected user base. We want this feature to be used.

— Strongly Favor

This is a very important feature for `mdspan` in the context of non-CPU accelerators like GPUs and FPGAs. I recently implemented such a feature in a library with similar capabilities to `mdspan` and with similar design choices (my library also has something like `extents`). It integrated well and proved beneficial.

— Strongly Favor

The application domains for `mdspan` definitely include contexts where the width of an index is an important performance consideration.

— Weakly Favor

Too late, no time to review it.

— Weakly Against

§ 3.1.6. [\[P2554R0\]](#) C-Array Interoperability Of `mdspan`

This paper is elegant in improving C++23 behaviour for immediate benefit, while also opening the door to extending `mdspan` capabilities if core language changes in the future.

— Strongly Favor

`mdspan` should support rank-1 C arrays.

— Strongly Favor

This removes a pitfall in `mdspan` initialization (providing the correct behavior in one case and trapping in another). It contains an obvious omission in the `Pointer&` deduction guide, but there's no design question there.

— Strongly Favor

This needs quick fixing and avoids a trap with `mdspan`. The paper seems a bit rushed though, like the paper number still being PXXXX. Furthermore, I am also surprised by the second deduction guide deducing a zero-dimensional `mdspan` from a pointer. This seems logical, but the paper lacks motivation for that change. Anyway, I still approve this fix.

— Weakly Favor

It seems like a win, but a small one. I'll still take it, but I feel we have to find a better solution overall.

— Weakly Favor

I consider raw arrays a complete waste of time, but I'm not opposed to this proposal.

— Neutral

This can be added later,

— Neutral

§ 3.1.7. [\[P2540R0\]](#) Empty Product For Certain Views

This improvement will increase coherence

— Strongly Favor

While the mathematically correct empty case for `std::zip` is impractical, that for `std::cartesian_product` is reasonable and therefore needful.

— Strongly Favor

This change makes an `cartesian_product` return a more sensible and mathematically consistent result.

— Weakly Favor

Makes sense, although I cannot be SF of this w/o wording.

— Weakly Favor

While the paper offers a compelling mathematical argument, it does not illustrate with example how the proposed change improves composition with other view, which is the question. It is not clear to me that there is a use case for a product of empty view either. As all the possible solutions do seem equally (un)motivated, this looks as reasonable as anything.

— Neutral

While this does have some mathematical basis, I am not convinced that this is a usability improvement for the adapter.

— Weakly Against

This paper does not have the wording. It is minimal but yet was not provided before the vote.

— Weakly Against

The paper/revision in question has no wording.

— Strongly Against

§ 3.1.8. [\[P2538R0\]](#) ADL-Proof projected

This restores parity between ranges and old versions of algorithms.

— Strongly Favor

Making the library more usable in edge cases is a good thing.

— Strongly Favor

This fixes an unfortunate case from the current specification

— Strongly Favor

This looks like an elegant solution to a complex problem, although it relies on a darker corner of the language.

— Weakly Favor

It's an unfortunate reality that types like `Holder *` trigger instantiation of the class - and hence hard errors - at the drop of a hat. This paper tries to work around one aspect of the problem - which is fine - but doesn't solve the whole problem.

— Weakly Favor

A general feature to control the set of associated entities (or one of the various "tame ADL" proposals) might be a better goal, but there's no harm done by implementing this carefully now (aside from yet-longer symbol names).

There is an R1, although it doesn't seem to contain any technical changes.

— Weakly Favor

§ 3.1.9. [\[LWG3646\]](#) `ranges::view_interface::size` Returns A Signed Type

All sizes should be unsigned for consistency

— Strongly Favor

We should apply the results of unsigned/signed discussions consistently.

— Strongly Favor

Whether or not sizes should be represented with unsigned types, at this point we should stay consistent with our established practice and fix outliers.

— Strongly Favor

I believe this generalizes the interface

— Weakly Favor

For better or worse, we've been defaulting to unsigned sizes for the ranges provided by the standard library, so we should be consistent.

— Weakly Favor

§ 3.1.10. [\[P2520R0\]](#) `move_iterator` Should Be A Random Access Iterator

Iterators category describe how iterators can move over a sequence, which is a property of the sequence itself, not its elements. As such, the proposed change makes perfect sense.

— Strongly Favor

This will make ranges code that uses moves better.

— Strongly Favor

Doing otherwise is inconsistent with status quo and other similar cases.

— Strongly Favor

Making `move_iterator` input doesn't seem to solve any problems - even input iterators can be dereferenced multiple times as long as the iterator isn't modified (or otherwise invalidated) - only creates new ones.

— Strongly Favor

If anyone wants to distinguish xvalue ranges (a prvalue range is already at best an `input_range` if it can't recompute a value), they're welcome to do so, but the `iterator_category` is the wrong tool for that purpose.

— Weakly Favor

§ 3.1.11. [\[P2499R0\]](#) `string_view` Range Constructor Should Be Explicit

Without this fix, there are many libraries and applications where it will not be possible to use `string_view`.

— Strongly Favor

The potential pitfall mentioned in the paper is very real. At the same time this range constructor is quite handy. When usability (reduced typing) and safety conflict we should prefer safety.

— Strongly Favor

This converting constructor is a silent semantic promotion. It should have been explicit to start with.

— Strongly Favor

I think this is the correct change, it preserves users to have simple syntax to convert to `string_view`, while avoiding surprising semantics.

— Strongly Favor

More explicit constructors is generally good, but I feel like explicitly converting to `string_view` is a nuisance. On balance I think the former wins out here.

— Weakly Favor

As the person who added that infamous constructor, I have concerns that

- This hurts composability of views - which can be mitigated by adding a generic adaptor that calls an explicit constructor in a generic manner (`view | std::view::as | ...`)
- We are ascribing to `string_view` and strings more meaning that they can possibly provide to provide. `string` and `vector` only differ by their names and a null-terminator, and the motivation for this paper - breakage in `fmt` - do call for a better solution. That `string_view` is displayed as a string, but `QLatin1String` isn't and `span` doesn't even compile is interesting, to say the least. Not sure that preventing a `vector` to be convertible to `string_view` is the solution.

That being said, by my own logic, `string_view` is redundant with `span` - the only salient difference is the constness of `string_view`, and there was very strong support for a change of direction, and very little support for my eccentric views on strings.

And this is miles better than just removing the constructor all together. Ship it.

— Weakly Favor

The confusion still exists, just not implicitly.

— Weakly Against

The poll fails to specify which option is being taken, although the minutes imply it's Option 1.

The potential pitfalls outlined in the paper are real, but the fact that `char*` is special (and that arrays can decay but `std::array` cannot) is hardly news to anyone doing string processing in C++.

Converting a `std::string` to a `std::string_view` doesn't stop at any included null, and so it should not surprise anyone that converting any other container behaves the same way.

— Weakly Against

§ 3.1.12. [\[P2549R0\]](#) `unexpected` Should Have error As Member Accessor

I believe `error()` is the better name as it makes the code clarity less dependent on the name of the object it's called on.

— Strongly Favor

It makes more sense for the accessor of a type that's meant to contain only errors to be `error()`. `value()` seems like a misnomer.

— Strongly Favor

It makes sense, but the status quo would have been manageable for my customers. As long as we pick one...

— Weakly Favor

Yes, this is slightly better, although i still don't understand the motivation for `unexpected` to have any public member

— Weakly Favor

I am convinced by the argument that `unexpected` is a type-safe cousin to `expected` where we statically know the contained value is an error.

— Weakly Favor

I'm not convinced by the rationale as I think it relies on false equivalences, but I don't really care

— Neutral

It can be argued that consistency is improved by this change, but it should be rare to use `std::unexpected` except as a tag and even rarer to use it in a generic context where the name really matters.

— Neutral

§ 3.1.13. [\[P2517R0\]](#) Add A Conditional `noexcept` Specification To `apply`

A very nice fix, the `noexceptness` of such facilities should preferably be transparent.

— Strongly Favor

We already applying conditional `noexcept` for most of the functional part of the standard library, and this makes it consistent.

— Strongly Favor

Missing a link in the conditional `noexcept` chain throws off the `noexcept-ness` of downstream consumers.

— Strongly Favor

The `zip` family should be implementable using `apply`.

— Weakly Favor

It's obviously correct for all the call wrappers to forward as many properties as possible (even if we wish the language made this easier).

— Weakly Favor

I'm not convinced by the motivation. The use of `apply` in the example doesn't seem materially different from the use of the `lambda`, which doesn't have a `noexcept` specification either.

This is close to mandating QoI. I see no real drawbacks but also not much point.

— Weakly Favor

§ 3.2. C++26 and Technical Specification Polls

§ 3.2.1. [\[P2300R5\]](#) `std::execution`

Getting this in early in the 26 cycle will make it available as the basis of further work and will give us time to find and fix any issues that arise.

— Strongly Favor

We need this as the basis for further work. Unless we settle on this (preferably in plenary) as the status quo we will "discuss in circles" forever.

— Strongly Favor

We really need this ASAP. It was unfortunate it missed C++23

— Strongly Favor

While the concerns about its readiness for C++23 were quite reasonable, it seems to have been established that this system is appropriately efficient and generic to be the foundation for C++26 structured concurrency even if almost all use will be by wrappers yet to be devised.

— Weakly Favor

I'm not seeing the point of sending this right now to languish in LWG's queue until next February.

— Neutral

I want to believe that `get_delegatee_scheduler` addresses one of my major objections to this paper: Lack of the ability to explicitly break cyclic graphs and thereby support reentrancy but the description/definition is so vague that I can't tell if it actually addresses this concern or I'm just reading into it what I want to see.

Moreover I continue to have concerns with the speed with which this paper is being pushed/forwarded: The version we're voting to forward wasn't even available through `wg21.link` when polling opened, for example.

— Weakly Against

I believe that further revision is needed in order to better correlate P2300 with the relevant papers:

- The `async_scope` proposal seems to be a better mechanism for spawning work than `ensure_started` and `start_detached`.
- It is not yet clear how the standard library algorithms will be overloaded to work with `std::execution`.
- The system execution context integration is not fully designed.

On the other hand, it seems that P2300's additions to the header (primarily, `tag_invoke`) would be better suited for a more general separate paper.

— Strongly Against

§ 3.2.2. [\[P1083R5\]](#) `resource_adaptor`

Adapting a non-polymorphic allocator to a pmr is functionality that ought to be put into the standard.

— Strongly Favor

The issue that LWG noted has been addressed and the reasons for moving this into the working draft remain.

— Strongly Favor

I can see the use case, we still need to finalize some of the details (which can be finalized by C++26's deadline)

— Weakly Favor

I do not have strong opinion, however I do want `std::aligned_raw_storage`, as an improvement over deprecated `std::aligned_storage`

— Weakly Favor

I need `aligned_raw_storage` like yesterday. `aligned_object_storage` seems not designed for polymorphic types.

— Weakly Favor

As stated, this is missing the changes suggested during the LEWG review. In particular, it's not clear that the design of `aligned_object_storage` is the right one.

— Weakly Against

Not sure the design is right yet, it's still in flux.

— Weakly Against

This facility should not be standardized. Its claims about what is or is not a vocabulary type are questionable, and `polymorphic_allocator` has loads of problems in and of itself, and turning actual allocators into memory resources has more. The standard doesn't need to and shouldn't encourage hacks like this, it should rather suggest that allocator-aware types be written as templates, so that they are actually customizable wrt. allocation, which memory resources or single-never-fits-all non-vocabulary types do not achieve.

— Strongly Against

This revision R5 up for vote does not reflect LEWG guidance from the forwarding poll; the forwarding poll was conditional on implementing such guidance.

— Strongly Against

§ 3.2.3. [P1202R4] Asymmetric Fences

I believe that the benefits of this additional synchronization primitives need to be confirmed, and TS is correct vehicle for them.

— Strongly Favor

The design here is simple and satisfactory, and already unanimously approved before. The only changes are those dictated by P2396, which was also unanimously approved. This is useful functionality for the TS.

— Strongly Favor

Despite my reservations about library TSs, I think we should get wider experience and feedback with asymmetric fences and the TS is not inappropriate.

— Strongly Favor

I want this feature to make progress into TS2, even though I do not fully understand its uses

— Weakly Favor

It seems reasonable, but I was not there for the discussions so I cannot take a stronger stance.

— Weakly Favor

No mention of implementation experience, or what is required (is it a pure library proposal? Presumably not if it will require changes to the compiler's memory model?)

— Weakly Against

§ References

§ Informative References

[LWG3646]

Jiang An. *[std::ranges::view_interface::size returns a signed type](https://wg21.link/lwg3646)*. LEWG. URL: <https://wg21.link/lwg3646>

[P0592R4]

Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: <https://wg21.link/p0592r4>

[P0792R8]

Vittorio Romeo, Zhihao Yuan, Jarrad Waterloo. *function_ref: a non-owning reference to a Callable*. 16 February 2022. URL: <https://wg21.link/p0792r8>

[P1083R5]

Pablo Halpern. *Move resource_adaptor from Library TS to the C++ WP*. 24 February 2022. URL: <https://wg21.link/p1083r5>

[P1169R3]

Barry Revzin, Casey Carter. *static operator()*. 14 October 2021. URL: <https://wg21.link/p1169r3>

[P1202R4]

David Goldblatt. *Asymmetric Fences*. 15 February 2022. URL: <https://wg21.link/p1202r4>

[P1223R3]

Zach Laine. *find_last*. 15 February 2022. URL: <https://wg21.link/p1223r3>

[P1885R10]

Corentin Jabot, Peter Brett. *Naming Text Encodings to Demystify Them*. 14 February 2022. URL: <https://wg21.link/p1885r10>

[P2300R5]

Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach. *std::execution*. 22 April 2022. URL: <https://wg21.link/p2300r5>

[P2499R0]

James Touton. *string_view range constructor should be explicit*. 7 December 2021. URL: <https://wg21.link/p2499r0>

[P2517R0]

Hewill Kang. *Add a conditional noexcept specification to std::apply*. 14 January 2022. URL: <https://wg21.link/p2517r0>

[P2520R0]

Barry Revzin. *move_iterator should be a random access iterator*. 16 January 2022. URL: <https://wg21.link/p2520r0>

[P2538R0]

Arthur O'Dwyer. *ADL-proof std::projected*. 15 February 2022. URL: <https://wg21.link/p2538r0>

[P2540R0]

Steve Downey. *Empty Product for certain Views*. 16 February 2022. URL: <https://wg21.link/p2540r0>

[P2549R0]

Yihe Li. *[std::unexpected should have error\(\) as member accessor](https://wg21.link/p2549r0)*. 13 February 2022. URL: <https://wg21.link/p2549r0>

[P2553R1]

Christian Trott, Damien Lebrun-Grandie, Mark Hoemmen, Dan Sunderland. *[Make mdspan size_type controllable](https://wg21.link/p2553r1)*. 16 March 2022. URL: <https://wg21.link/p2553r1>

[P2554R0]

Christian Trott, Damien Lebrun-Grandie, Mark Hoemmen, K. R. Walker, Dan Sunderland. *[C-Array Interoperability of MDSpan](https://wg21.link/p2554r0)*. 15 February 2022. URL: <https://wg21.link/p2554r0>

[P2574R0]

Bryce Adelstein Leibach. *[2022-05 Library Evolution Polls](https://wg21.link/p2574r0)*. 22 April 2022. URL: <https://wg21.link/p2574r0>